

CS 663: Digital Image Processing

Assignment 1

Addepalli Bhargava Sarma (24B0013)
T Sai Dheeraj (24B0960)

Contents

1	Image Subsampling and Interpolation	3
1.1	Image Shrinking	3
1.2	Image Enlargement using Nearest-Neighbor Interpolation	3
1.2.1	Result	4
1.3	Image Enlargement using Bilinear Interpolation	4
1.3.1	Result	4
1.4	Image Enlargement using Bicubic Interpolation	4
1.4.1	Result	5
1.5	Image Rotation using Interpolation	5
1.5.1	Rotation Angle	5
1.5.2	Bilinear Interpolation	6
1.5.3	Nearest-Neighbor Interpolation	6
1.5.4	Comparison	7
1.5.5	Result	7
1.6	Image Downsampling and Upsampling	7
1.6.1	Enlarged Images	7
1.6.2	Difference Images	8
1.6.3	RMSE	8
1.6.4	Inference from Enlarged Images	8
1.6.5	Inference from Difference Images	8
2	Thresholding	8
2.1	Manual Thresholding	9
2.1.1	Parameter Tuning	9
2.1.2	Receipt	9
2.1.3	Blackboard	10
2.1.4	Lilavati	11
2.1.5	QR	12
2.2	Otsu Thresholding	12
2.2.1	Receipt	13

2.2.2	Blackboard	14
2.2.3	Lilavati	15
2.2.4	QR	16
2.3	Local / Adaptive Thresholding	16
2.3.1	Parameter Tuning	16
2.3.2	Receipt	17
2.3.3	Blackboard	17
2.3.4	Lilavati	18
2.3.5	QR	18
3	Contrast Enhancement	18
3.1	Linear Contrast Stretching	18
3.1.1	Parameter Selection	19
3.2	Histogram Equalization	19
3.3	CLAHE – Canyon	20
3.3.1	Chosen Parameters	20
3.3.2	Baseline Result	21
3.3.3	Significantly Larger Window	21
3.3.4	Significantly Smaller Window	22
3.3.5	Half Histogram Threshold	23
3.4	CLAHE – Retina	24
3.4.1	Chosen Parameters	24
3.4.2	Baseline Result	25
3.4.3	Significantly Larger Window	25
3.4.4	Significantly Smaller Window	26
3.4.5	Half Histogram Threshold	27
3.5	Histogram Matching	28
3.5.1	Significantly Smaller Number of Bins	28
3.5.2	Chosen Number of Bins	29
3.5.3	Significantly Larger Number of Bins	30

1 Image Subsampling and Interpolation

1.1 Image Shrinking

Input image: data/interp/suit.png

Function: myImageShrink()

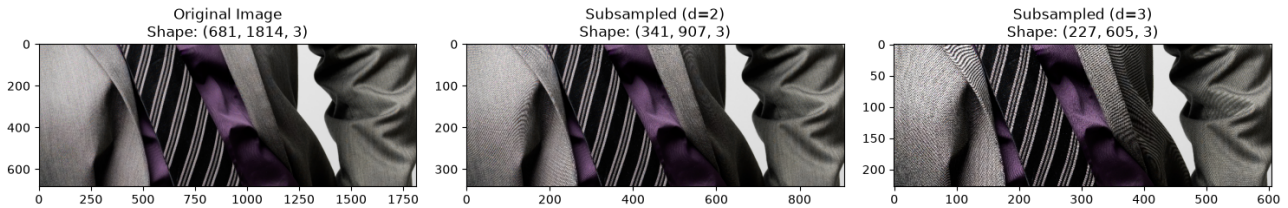


Figure 1: Original and subsampled images

Inference: Moire Effects in Subsampled Images

The moire patterns seen in the subsampled images arise from **aliasing**. The tie and suit fabric contain fine, high-frequency periodic structures (stripes, weave), and `myImageShrink()` performs simple point sampling (selecting every d -th pixel) with no low-pass pre-filtering. This violates the Nyquist criterion $f_s \geq 2f_{\max}$, so the high-frequency content folds back into low-frequency beat patterns, visible as wavy fringes. As d increases from 2 to 3, the effective sampling rate drops further, so more of the stripe frequencies alias, making the moire pattern stronger and shifting its apparent orientation/period.

1.2 Image Enlargement using Nearest-Neighbor Interpolation

Input image: data/interp/random.png

Function: myNearestNeighborInterpolation()

1.2.1 Result

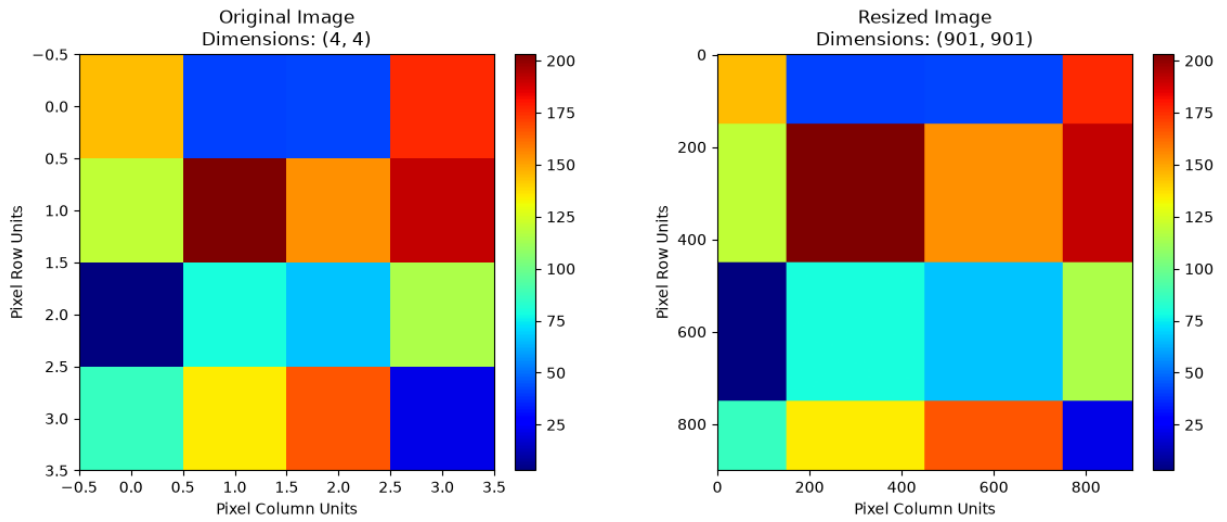


Figure 2: Original and nearest-neighbor interpolated images.

1.3 Image Enlargement using Bilinear Interpolation

Function: `myBilinearInterpolation()`

1.3.1 Result

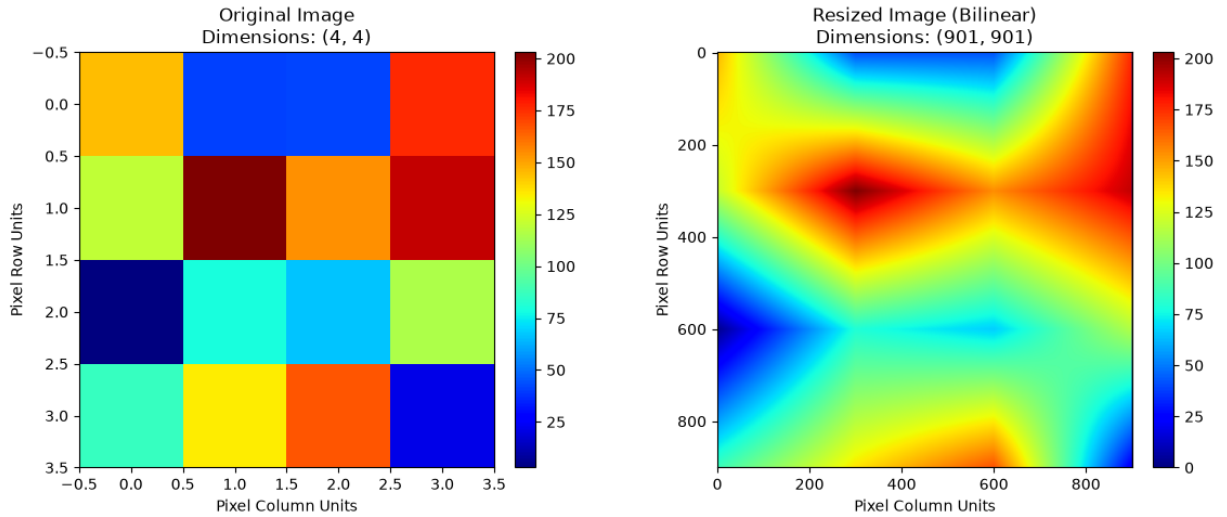


Figure 3: Original and bilinear interpolated images.

1.4 Image Enlargement using Bicubic Interpolation

Function: `myBicubicInterpolation()`

1.4.1 Result

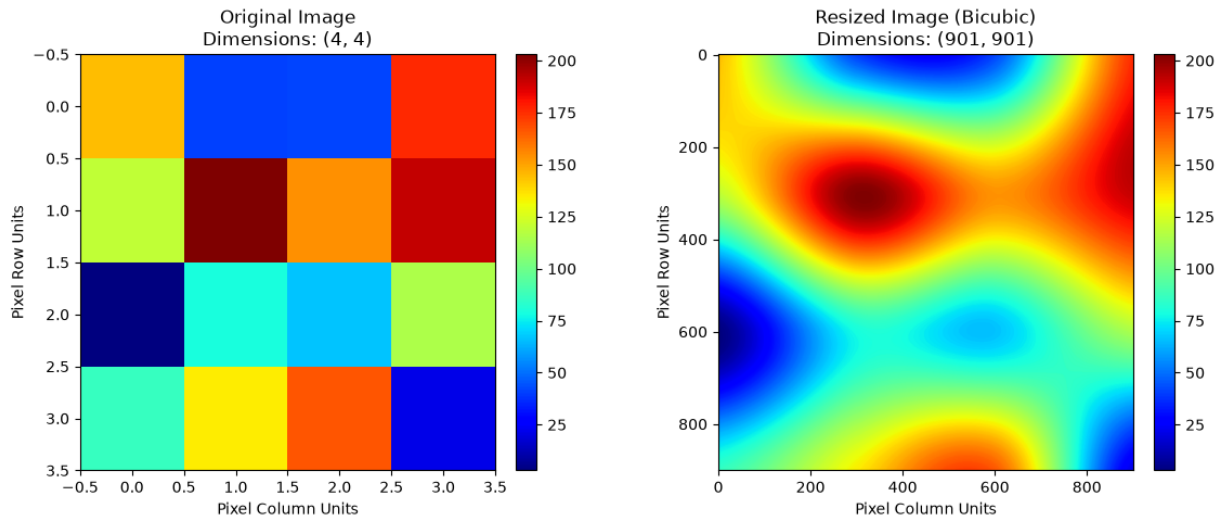


Figure 4: Original and bicubic interpolated images.

1.5 Image Rotation using Interpolation

Input image: data/interp/main.png

Functions:

- `myImageRotationUsingBilinearInterp()`
- `myImageRotationUsingNearestNeighborInterp()`

1.5.1 Rotation Angle

Rotation angle: $\tan^{-1}\left(\frac{14}{160}\right)$

We found the rotation angle from imageonline.io/find-coordinates-of-image/ this website, which lets you see the co-ordinates of pixels which can be picked roughly.

1.5.2 Bilinear Interpolation



Figure 5: Original and rotated image using bilinear interpolation.

1.5.3 Nearest-Neighbor Interpolation



Figure 6: Original and rotated image using nearest-neighbor interpolation.

1.5.4 Comparison

Comparing the two rotated images, the bilinear-interpolated result is visibly smoother and perceptually more pleasing than the nearest-neighbor result. In the nearest-neighbor image, straight edges (e.g., the vertical facade fins and window ledges of the building) exhibit a pronounced staircase / jagged (blocky) artifact, since each output pixel simply copies the value of its nearest source pixel with no blending across neighbors. In contrast, bilinear interpolation computes a weighted average of the four surrounding pixels, which smooths out these staircase artifacts and produces visually continuous, anti-aliased edges. The trade-off is that bilinear interpolation introduces a mild blurring of fine details and sharp transitions, whereas nearest-neighbor preserves sharper (but jagged) edges and original intensity values without introducing any new interpolated values. Overall, for this rotation task, bilinear interpolation gives a qualitatively better and more natural-looking result.

1.5.5 Result



Figure 7: Comparison of Bilinear and Nearest Neighbor rotation

1.6 Image Downsampling and Upsampling

Input: data/interp/ct.mat

The subsampled CT image was enlarged using nearest-neighbor, bilinear, and bicubic interpolation.

1.6.1 Enlarged Images

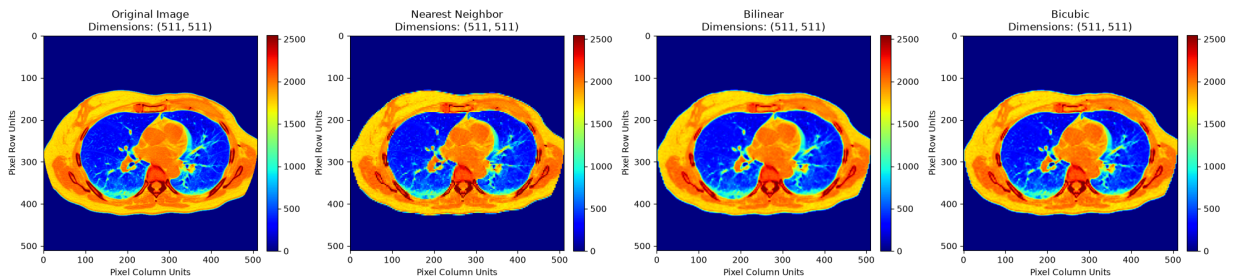


Figure 8: Original CT image and the three interpolated images.

1.6.2 Difference Images

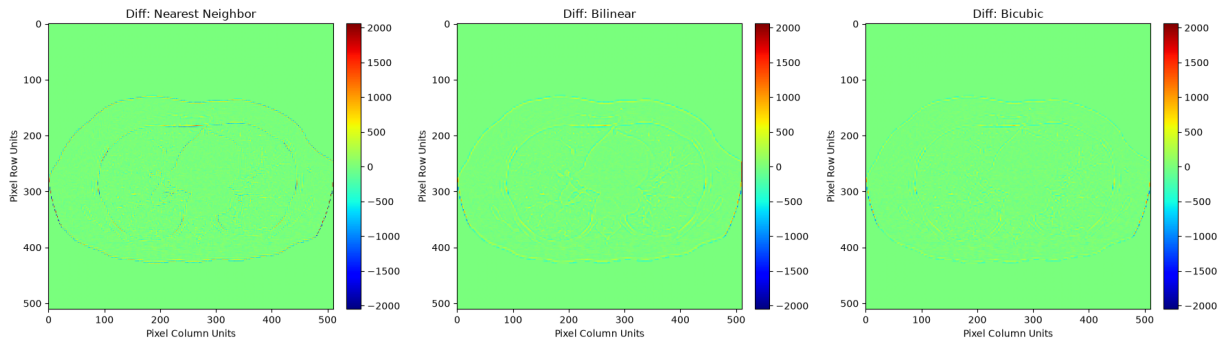


Figure 9: Difference images: original minus each interpolated image.

1.6.3 RMSE

Interpolation Method	RMSE
Nearest Neighbor	103.42
Bilinear	72.00
Bicubic	62.76

1.6.4 Inference from Enlarged Images

Visually, all three enlarged images appear qualitatively similar to the original at first glance, but closer inspection reveals differences in smoothness and edge fidelity. The nearest-neighbor result shows slight blockiness/jaggedness along curved edges since it simply replicates pixel values without any smoothing. The bilinear result is visibly smoother, as it linearly averages neighboring pixels, but this also introduces mild blurring at sharp anatomical boundaries. The bicubic result best preserves both smoothness and edge sharpness simultaneously, since it uses a larger neighborhood and higher-order polynomial fitting, producing the closest visual match to the original image.

1.6.5 Inference from Difference Images

The difference images (original – enlarged) confirm that errors are not spread uniformly but are concentrated almost entirely along high-frequency regions, i.e., edges and boundaries such as the ribs, lung contours, and skin outline, while flat/homogeneous regions show near-zero error across all three methods. The nearest-neighbor difference image shows the highest-magnitude and most widespread error along these edges, consistent with its blocky, non-smooth interpolation. The bilinear difference image shows reduced error magnitude but a similar edge-concentrated pattern, reflecting the blurring trade-off of linear averaging. The bicubic difference image shows the faintest and thinnest error bands, indicating it reconstructs edge regions most accurately among the three, which is consistent with it achieving the lowest RMSE (62.76) compared to bilinear (72.00) and nearest neighbor (103.42).

2 Thresholding

The input images are:

- `receipt.png`
- `blackboard.png`

- `lilavati.tif`
- `qr.png`

2.1 Manual Thresholding

Function: `manual_thresholding()`

2.1.1 Parameter Tuning

Image	Threshold T
Receipt	120
Blackboard	65
Lilavati	115
QR	110

2.1.2 Receipt

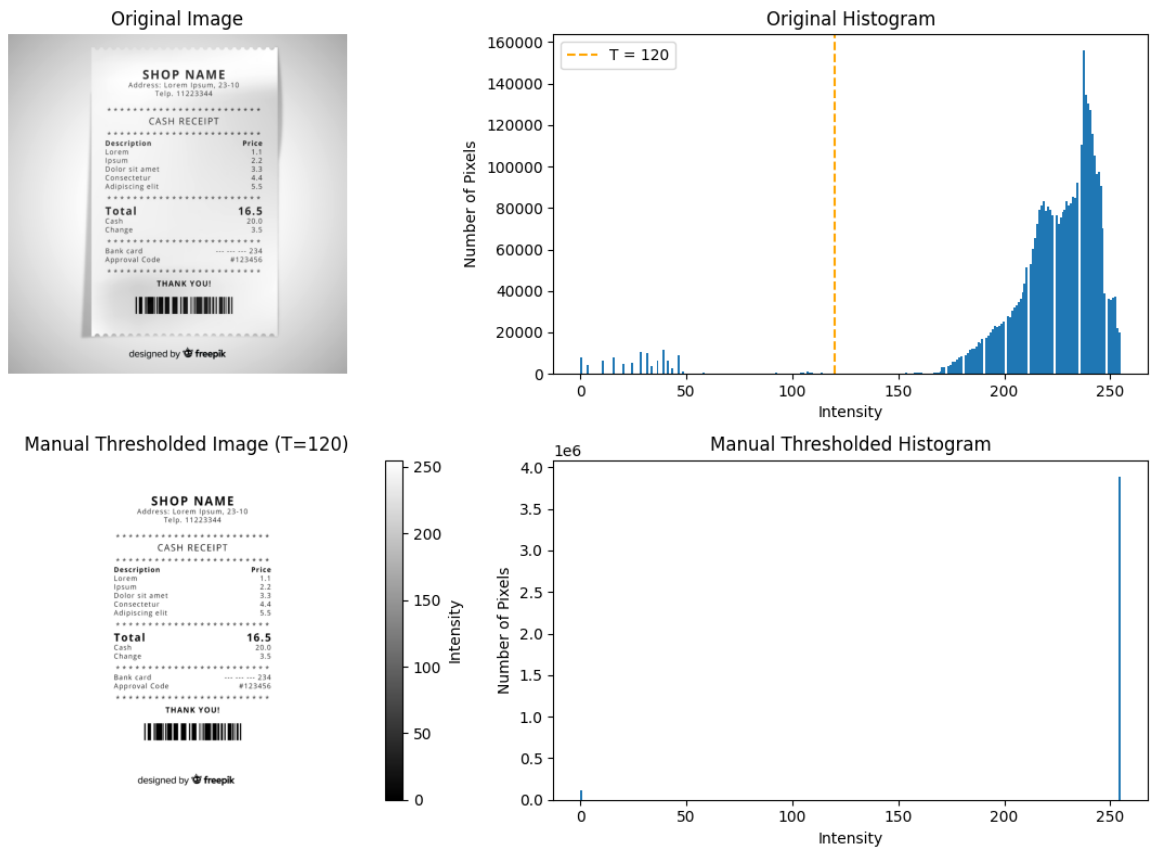


Figure 10: Manual thresholding of receipt.png.

Explanation: The receipt has a bright background and dark text, with most pixels concentrated at high intensities. A threshold of $T=120$ effectively separates the dark text from the bright background, producing a clear binary image with the receipt content preserved.

2.1.3 Blackboard

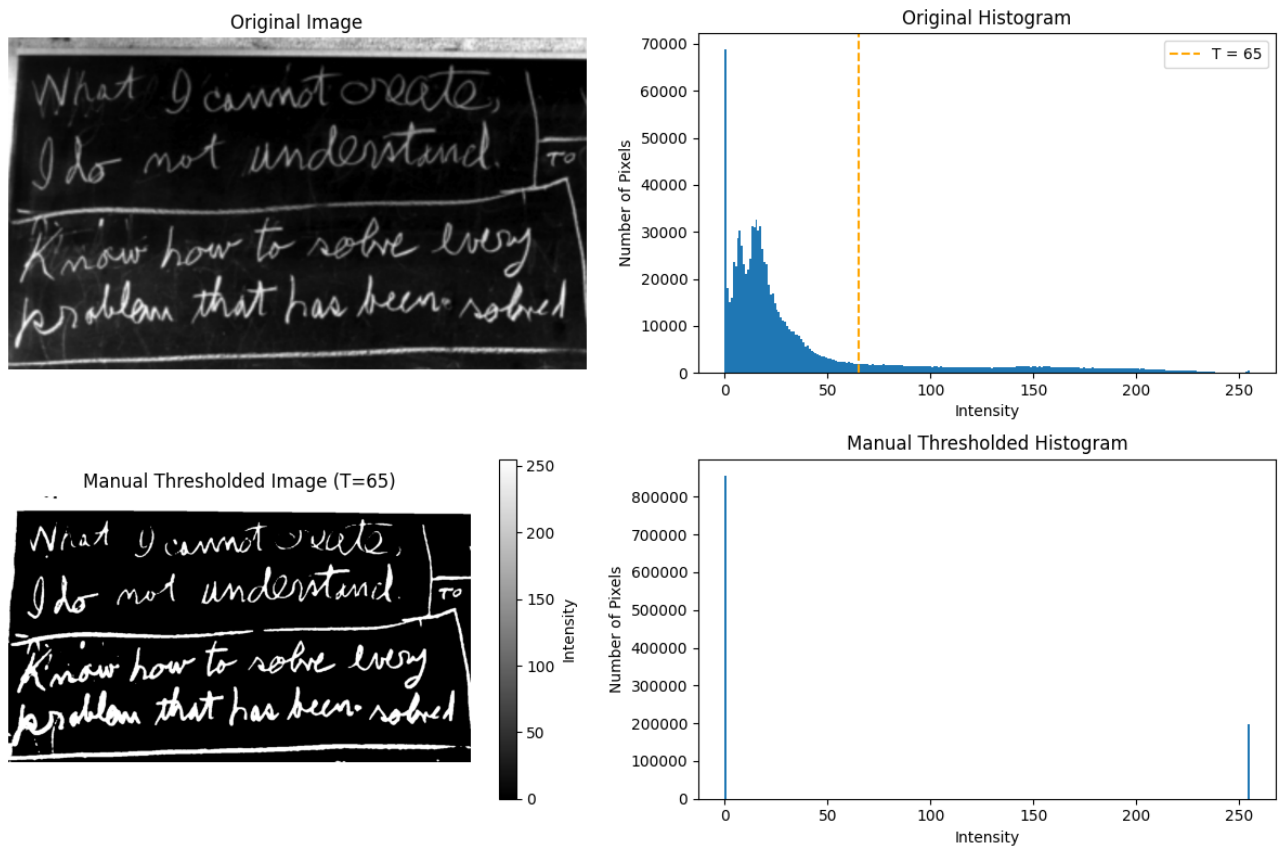


Figure 11: Manual thresholding of blackboard.png.

Explanation: The blackboard is predominantly dark, while the writing is significantly brighter. A relatively low threshold of $T=65$ separates the bright chalk strokes from the dark board effectively, preserving most of the writing while keeping the background black.

2.1.4 Lilavati

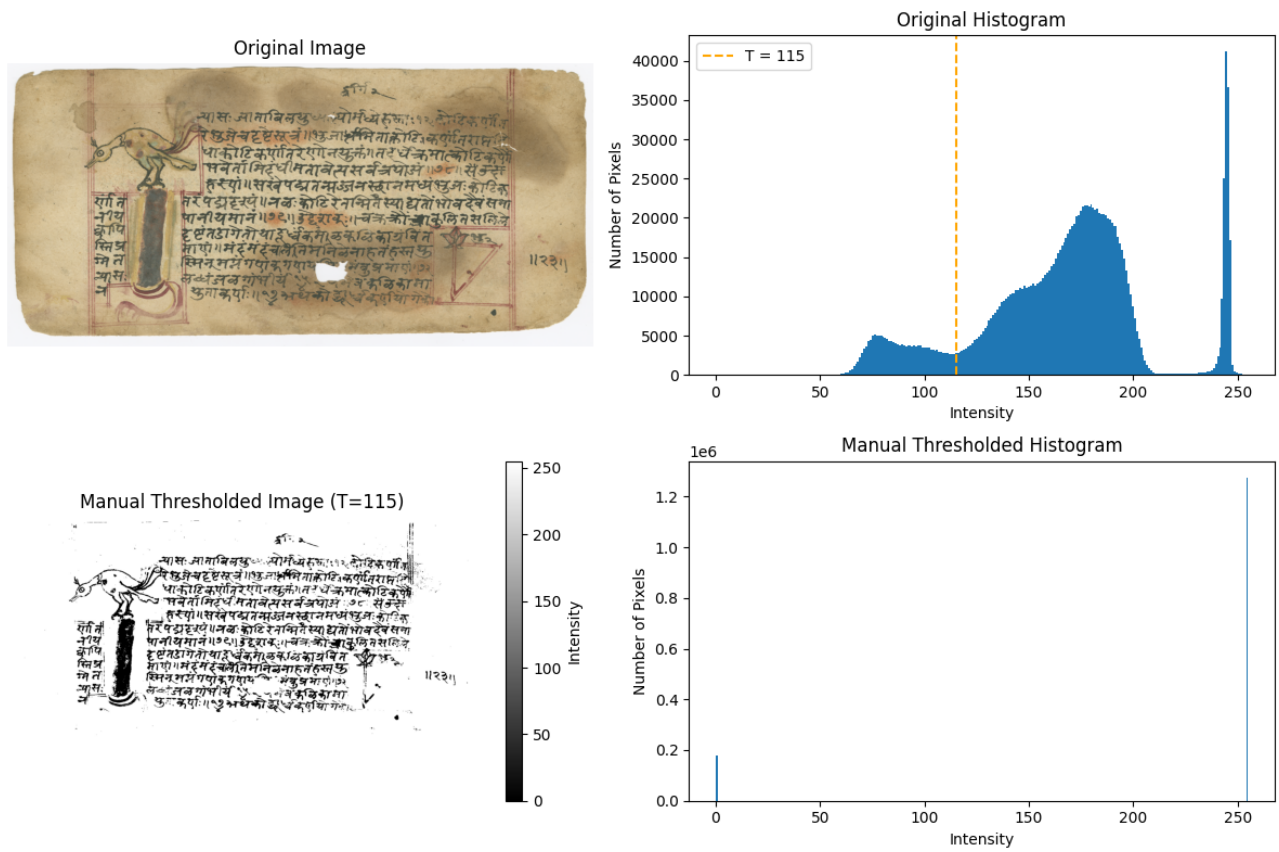


Figure 12: Manual thresholding of lilavati.tif.

Explanation: The Lilavati image contains a bright, uneven parchment background and darker text/illustrations. $T=115$ provides a reasonable separation between the dark writing and the lighter background, although variations in the parchment intensity cause some background regions to appear as foreground.

2.1.5 QR

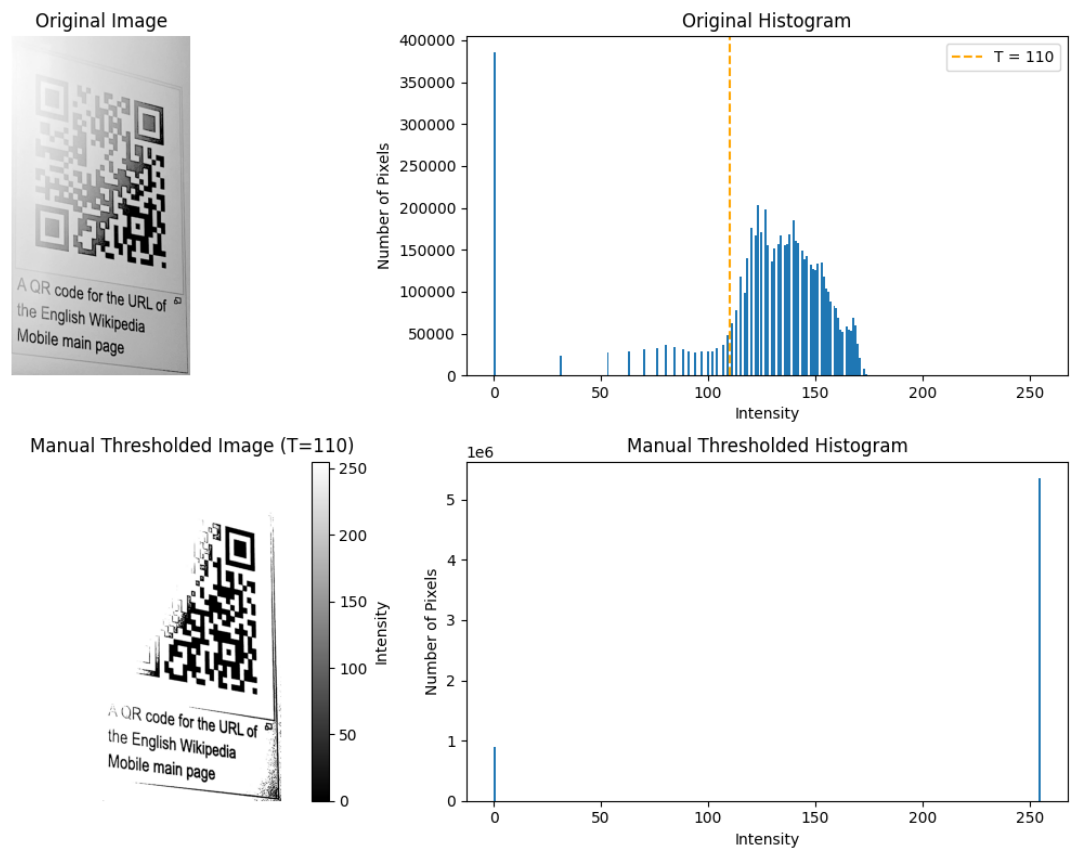


Figure 13: Manual thresholding of qr.png.

Explanation: The QR image has a bright background with dark QR patterns and text. A threshold of $T=110$ separates the dark QR modules and text from the bright background, producing a recognizable binary result. The perspective and varying illumination make the separation less uniform in some regions.

2.2 Otsu Thresholding

Function: `otsu_thresholding()`

2.2.1 Receipt

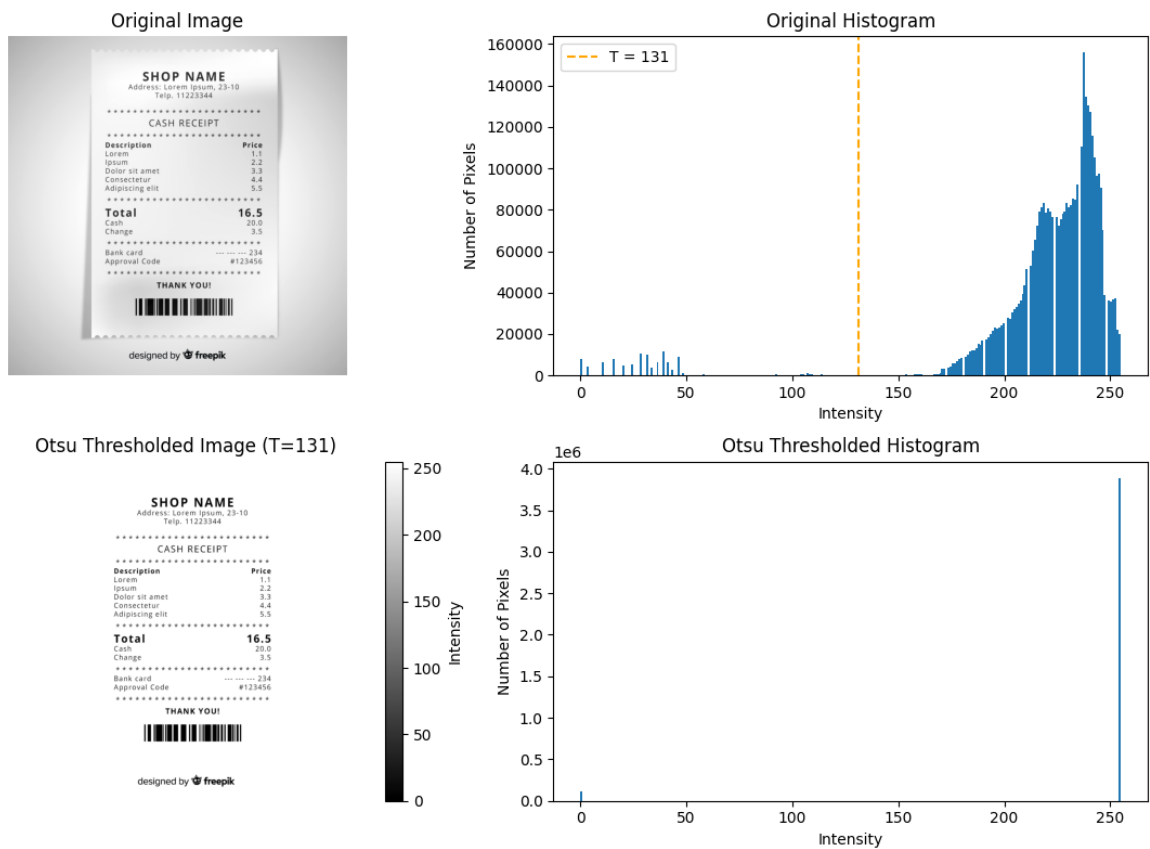


Figure 14: Otsu thresholding of receipt.png.

Explanation: Otsu works very well for the receipt because there is a strong separation between the dark text and the bright background. The threshold of 131 produces a clean binary image in which the text and barcode are preserved while most of the background is removed.

2.2.2 Blackboard

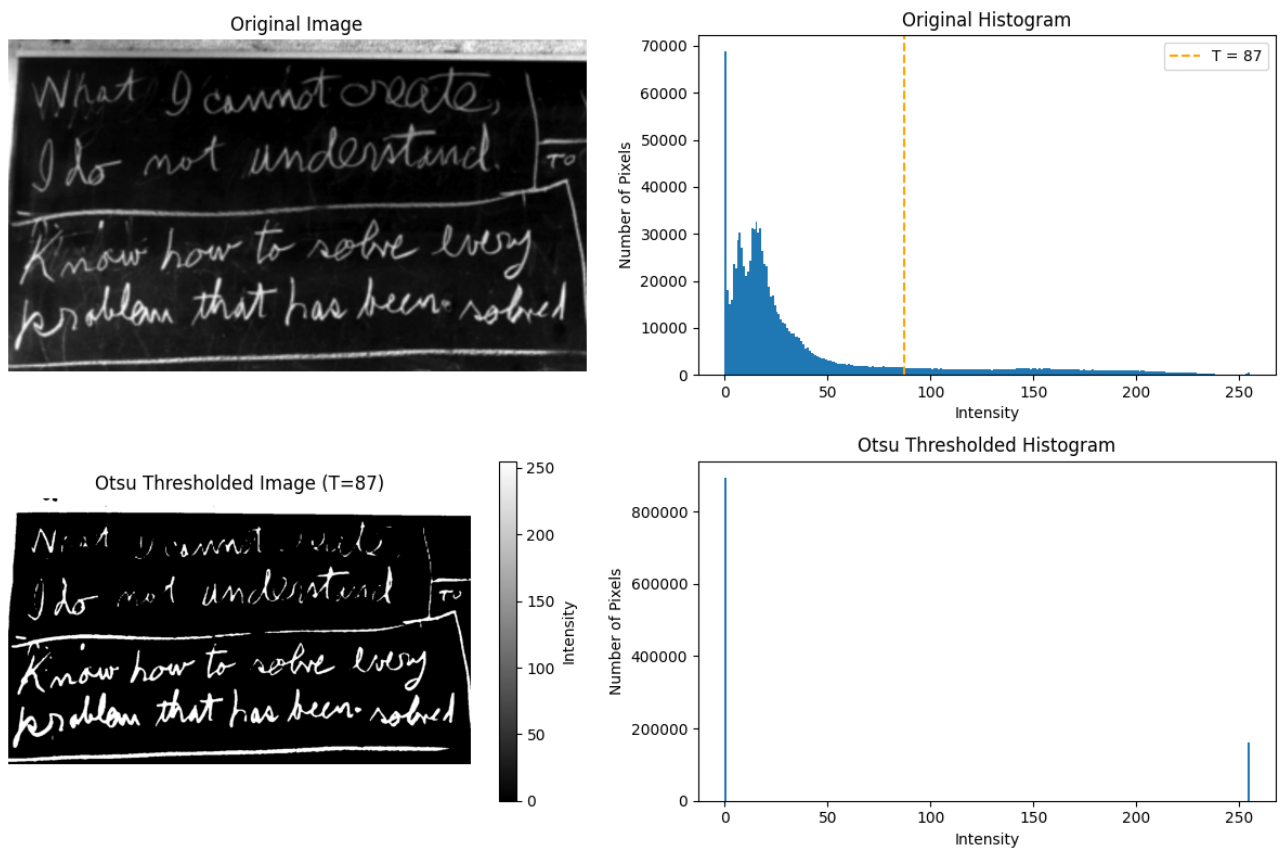


Figure 15: Otsu thresholding of blackboard.png.

Explanation: Otsu performs reasonably well because the blackboard and bright chalk writing form two relatively distinct intensity groups. The threshold of 87 preserves most of the writing and produces a clear binary image, although some thin/low-contrast strokes are lost.

2.2.3 Lilavati

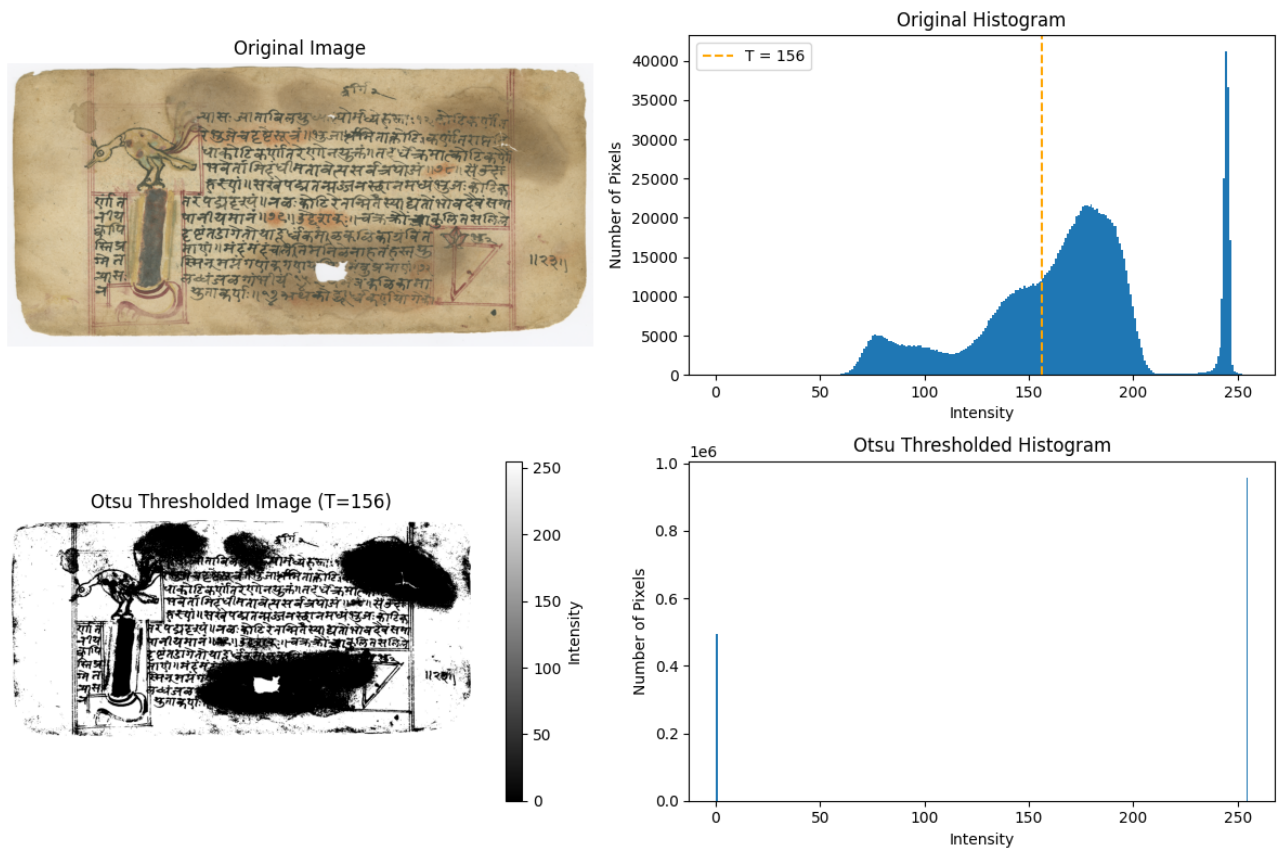


Figure 16: Otsu thresholding of lilavati.tif.

Explanation: Otsu performs poorly for this image. The parchment has substantial illumination and intensity variation, while the dark text is spread across a broad intensity range. The high threshold of 156 causes large dark/medium regions of the parchment to be classified as foreground, producing significant noise and unwanted regions.

2.2.4 QR

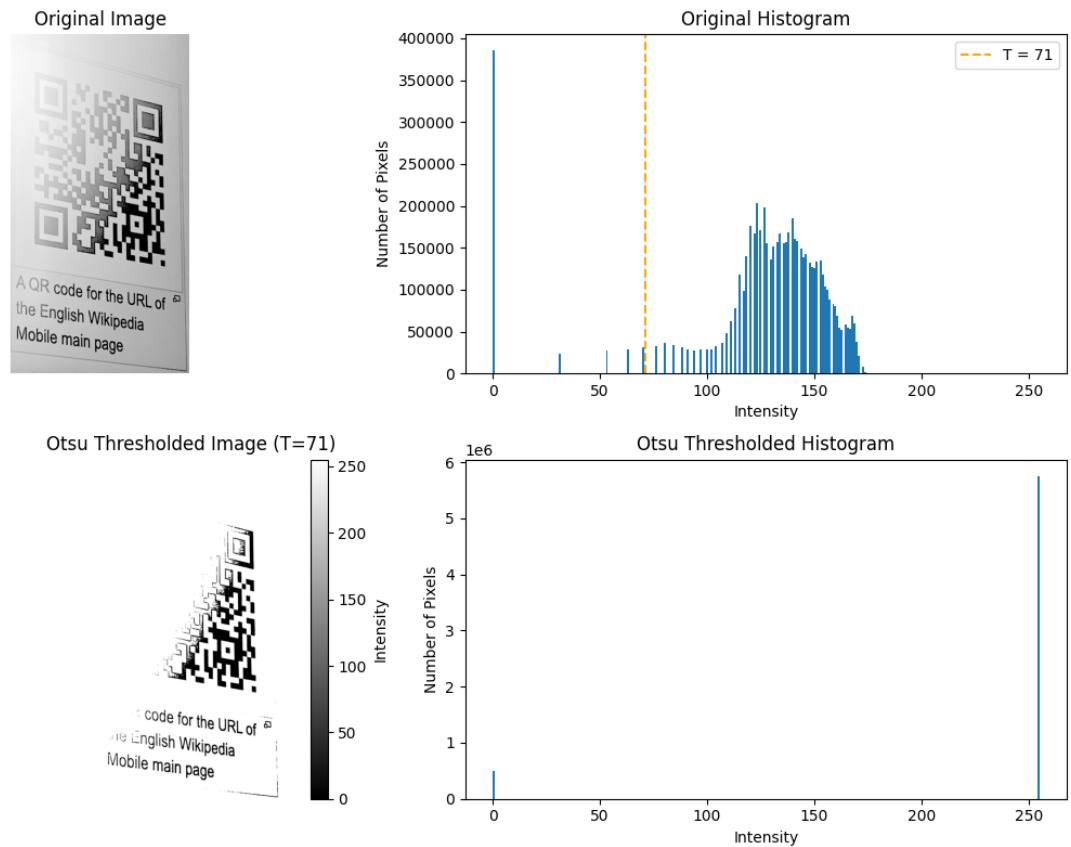


Figure 17: Otsu thresholding of qr.png.

Explanation: Otsu performs poorly on the QR image because the intensity distribution is not cleanly separated into two classes. The threshold of 71 causes much of the light QR/background region to be classified as foreground, resulting in loss of parts of the QR code and text.

2.3 Local / Adaptive Thresholding

Function: `adaptive_thresholding()`

2.3.1 Parameter Tuning

Image	Window Size	k	Method
Receipt	51	0.1	Sauvola
Blackboard	75	0.05	Sauvola
Lilavati	81	0.1	Sauvola
QR	51	0.03	Sauvola

2.3.2 Receipt

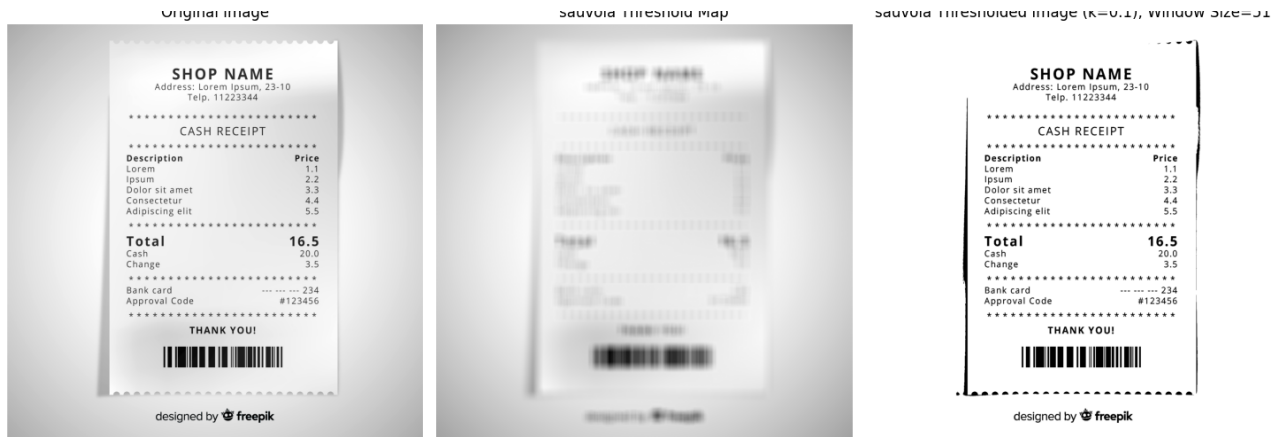


Figure 18: Adaptive thresholding and per-pixel threshold map for receipt.png.

Explanation: Adaptive thresholding works very well because the receipt contains a bright background with relatively dark text and barcode, along with mild illumination variation. The local threshold preserves the text and barcode clearly while removing most of the background. Some dark artifacts remain along the receipt edges.

2.3.3 Blackboard



Figure 19: Adaptive thresholding and per-pixel threshold map for blackboard.png.

Explanation: Adaptive thresholding works very well because the blackboard has uneven illumination and the chalk intensity varies across the image. The local threshold adapts to these changes and preserves most of the faint chalk writing while keeping the dark background black. A few small noisy regions remain near the borders.

2.3.4 Lilavati

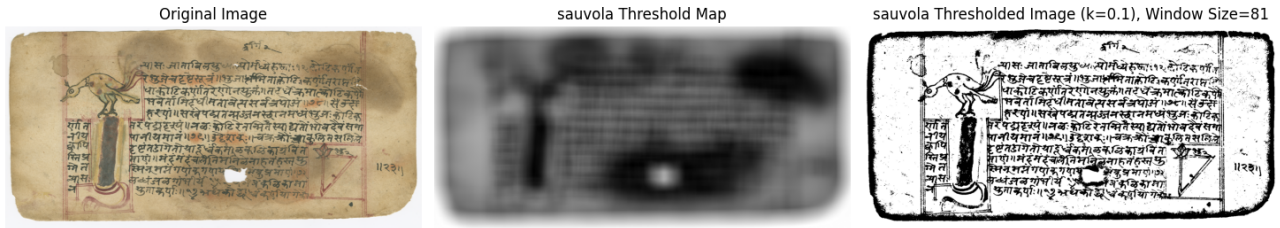


Figure 20: Adaptive thresholding and per-pixel threshold map for lilavati.tif.

Adaptive thresholding works well because the manuscript has significant background and illumination variation. The local threshold allows the dark text to be extracted across the non-uniform parchment. Most of the writing is preserved, although some dark stains and background regions are also classified as foreground.

2.3.5 QR



Figure 21: Adaptive thresholding and per-pixel threshold map for qr.png.

Explanation: Adaptive thresholding works very well for the QR image because the illumination varies across the tilted surface. The smaller window allows the threshold to adapt to local intensity changes, preserving the QR pattern and the text. Some small isolated noise is visible around the QR code and image boundaries.

3 Contrast Enhancement

3.1 Linear Contrast Stretching

Input image: data/hist/leh.png

Function: myLinearContrastStretch()

3.1.1 Parameter Selection

Parameter	Value
r_1	50
s_1	200

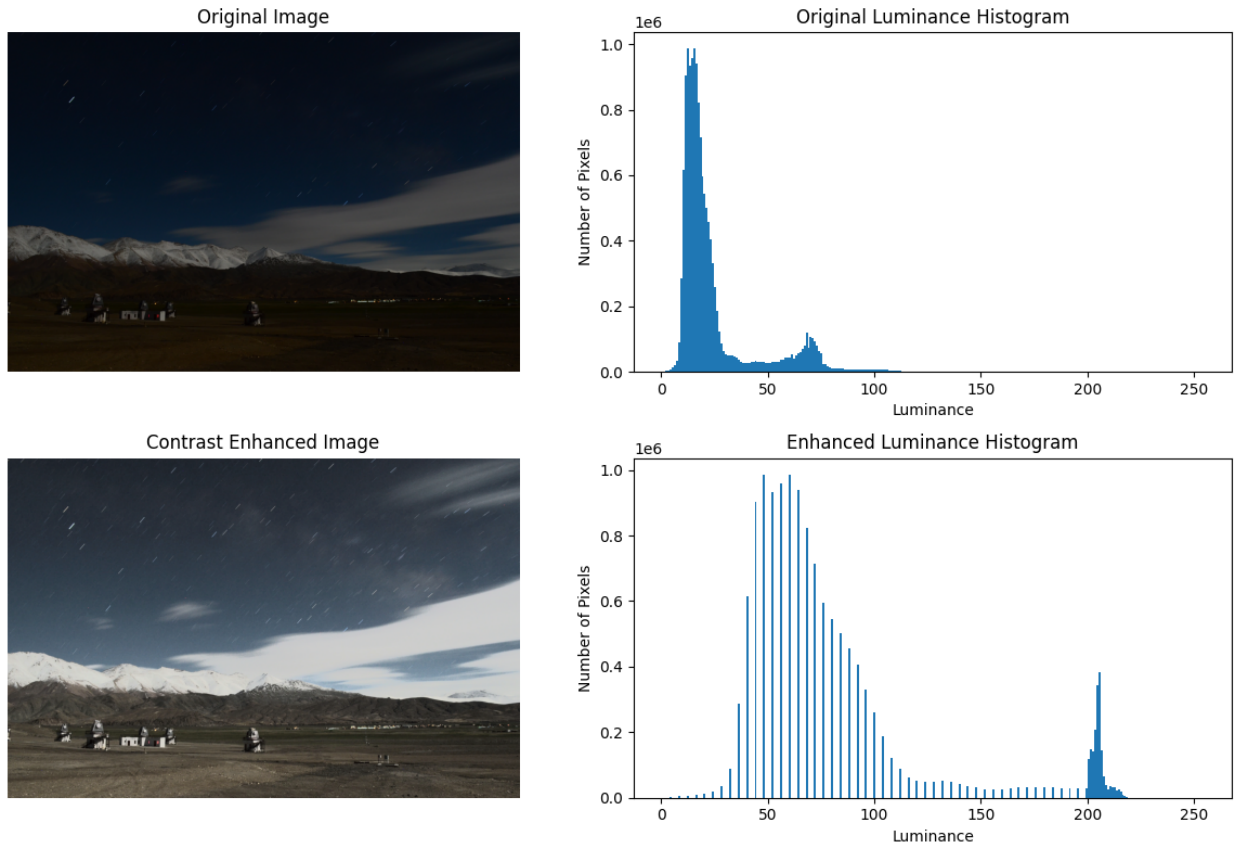


Figure 22: Linear contrast stretching: original/enhanced images and luminance histograms.

3.2 Histogram Equalization

Input image: data/hist/leh.png

Function: myHistEqualize()

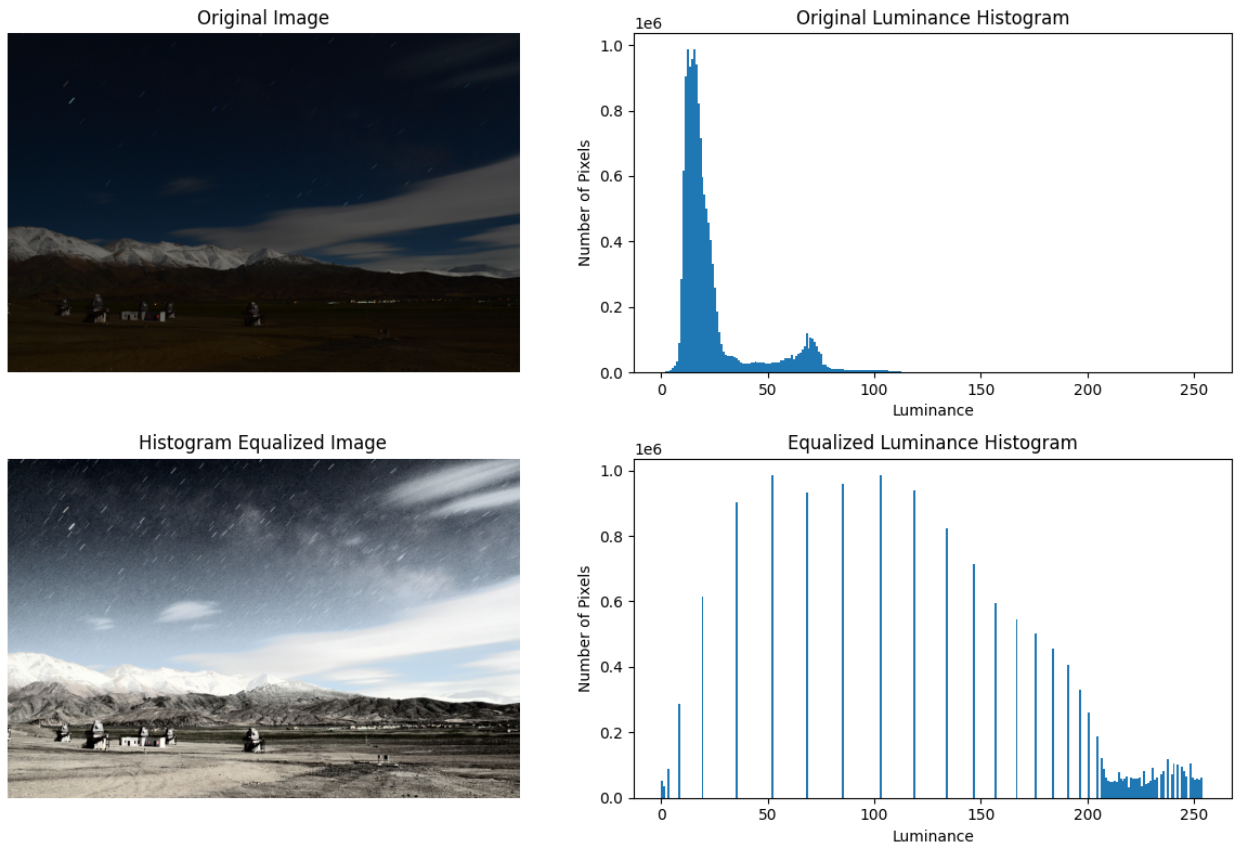


Figure 23: Histogram equalization: original/equalized images and luminance histograms.

3.3 CLAHE – Canyon

Input image: data/hist/canyon.png

Function: myCLAHE()

3.3.1 Chosen Parameters

Parameter	Chosen Value
Window size	71
Number of bins	256
Histogram threshold	0.03

3.3.2 Baseline Result

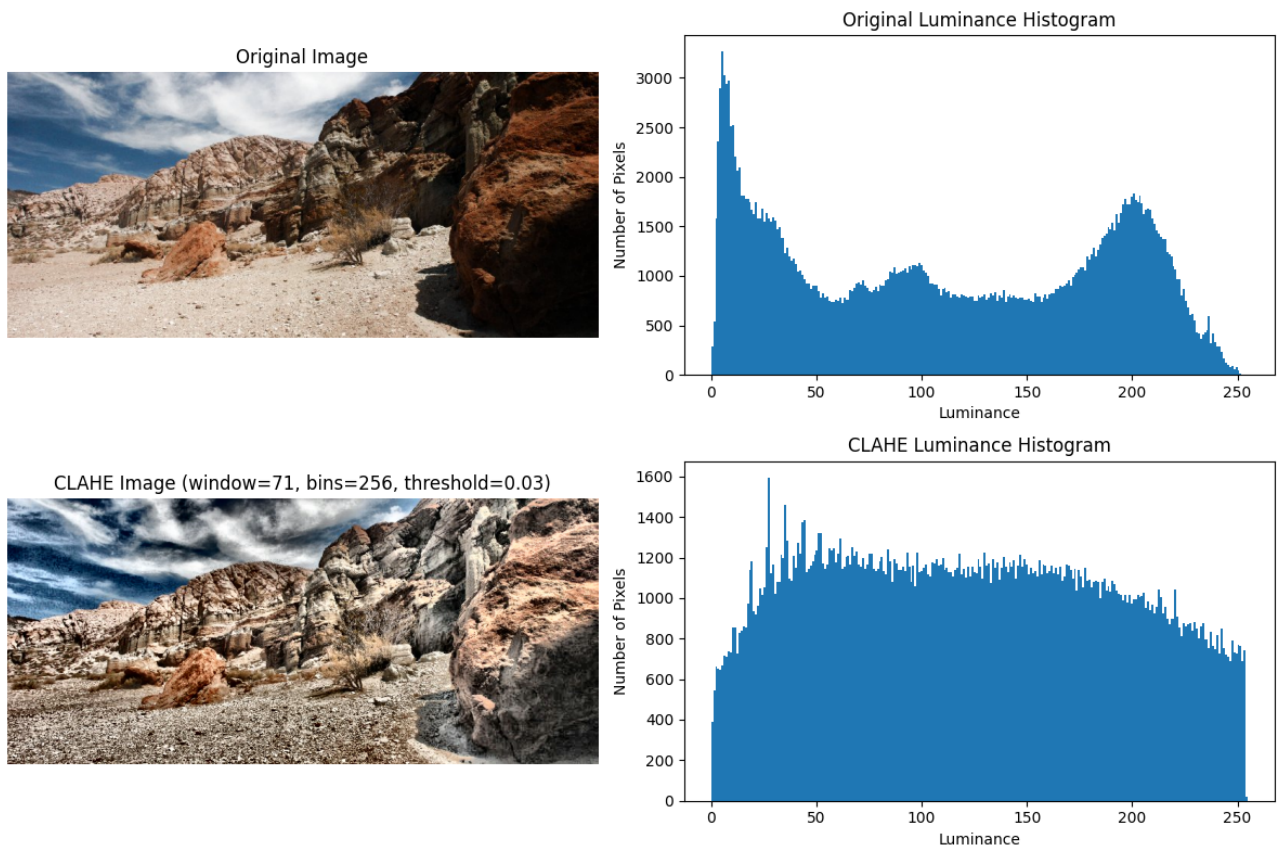


Figure 24: CLAHE on canyon.png using the tuned parameters.

Explanation: The selected parameters provide a good balance between local contrast enhancement and noise suppression. The dark shadowed regions, especially near the bottom-right boulder, become clearer while preserving the overall appearance of the image.

3.3.3 Significantly Larger Window

Window size: 151

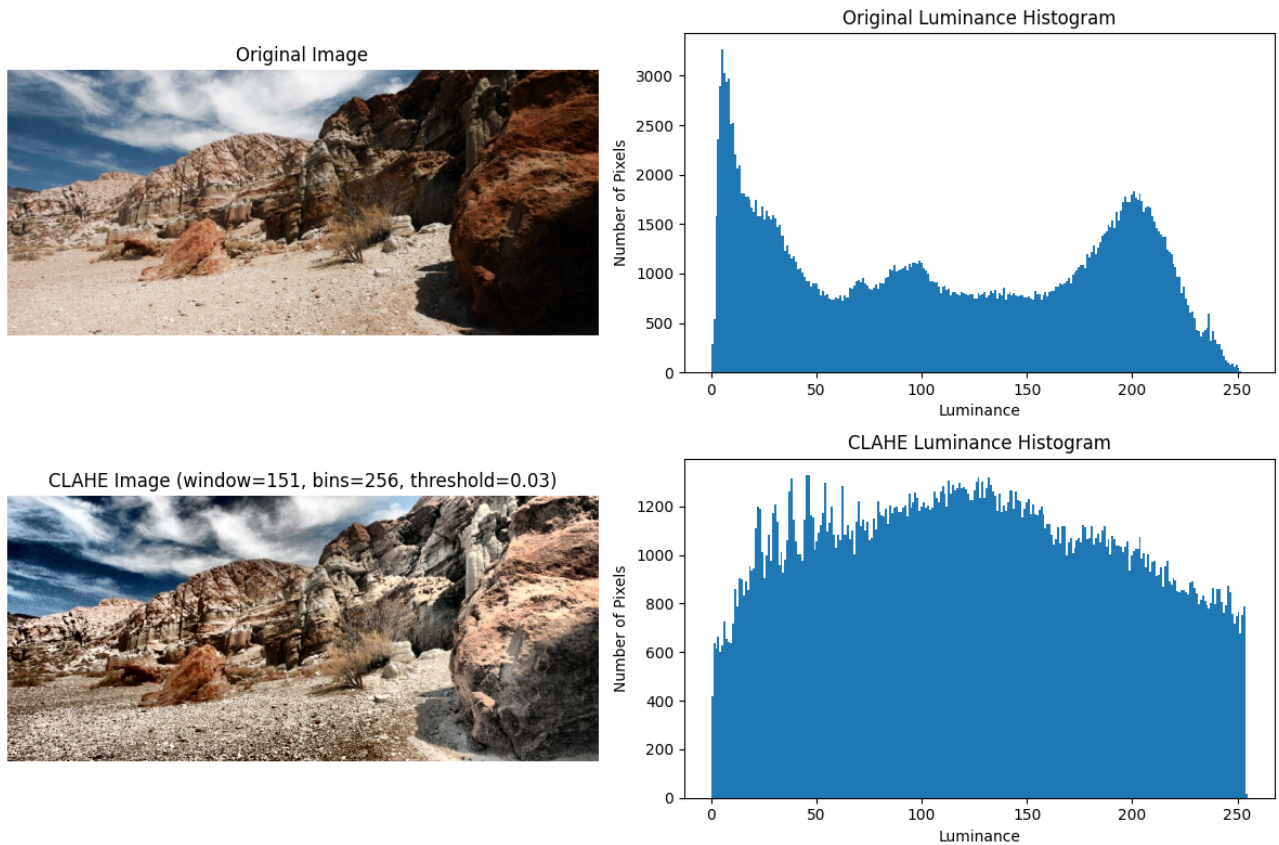


Figure 25: CLAHE on canyon.png using a significantly larger window.

Observation: The larger neighborhood produces weaker local enhancement because the histogram represents a broader region. The image appears smoother, but some local details and shadow regions are less enhanced.

3.3.4 Significantly Smaller Window

Window size: 31

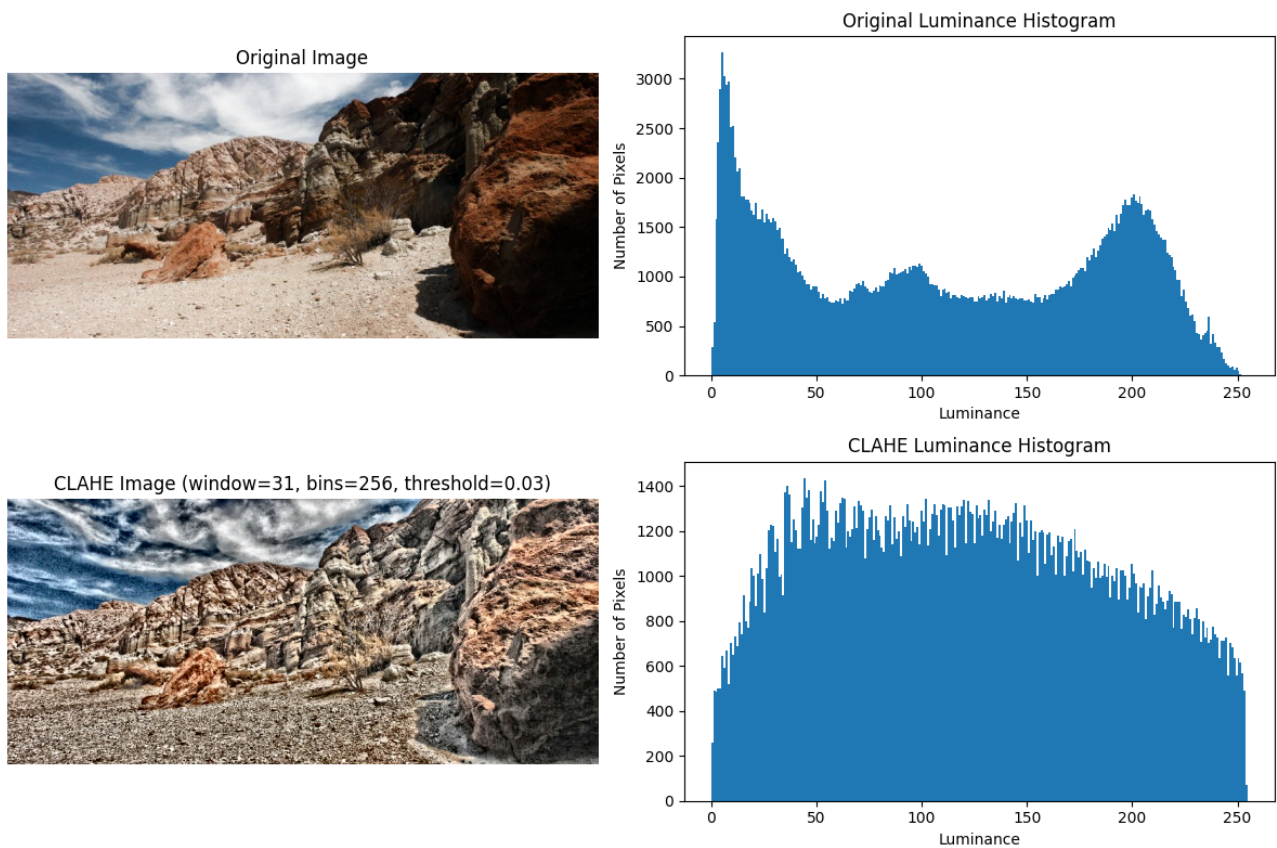


Figure 26: CLAHE on canyon.png using a significantly smaller window.

Observation: The smaller neighborhood provides stronger local contrast enhancement and makes fine details more visible. However, it also increases noise and local intensity variations, resulting in a more aggressive appearance.

3.3.5 Half Histogram Threshold

Window size: 71

Original threshold: 0.03

New threshold: 0.015

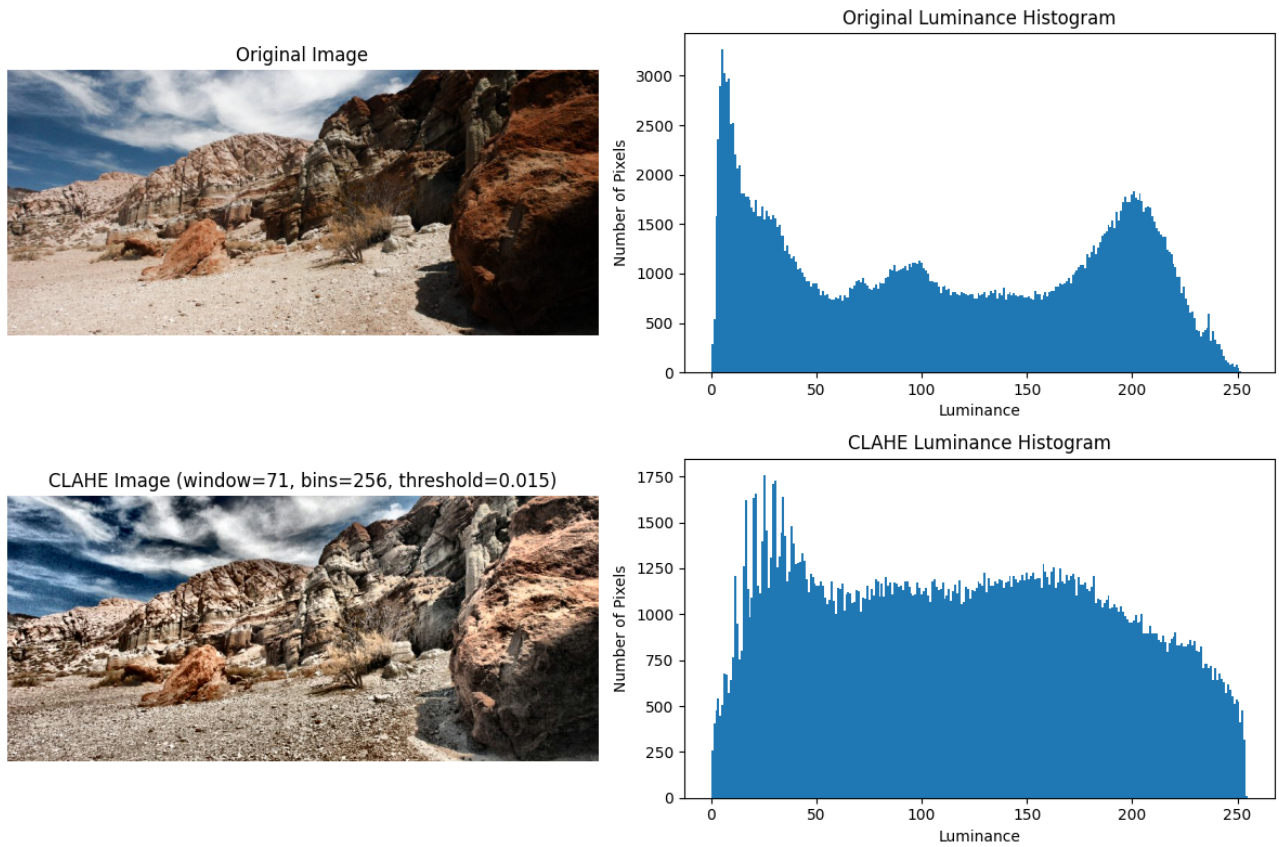


Figure 27: CLAHE on canyon.png using half the tuned histogram threshold.

Observation: Reducing the threshold increases histogram clipping and therefore produces stronger contrast enhancement. More details become visible in the darker regions, but there is also a greater risk of amplifying noise.

3.4 CLAHE – Retina

Input image: data/hist/retina.png

Function: myCLAHE()

3.4.1 Chosen Parameters

Parameter	Chosen Value
Window size	71
Number of bins	256
Histogram threshold	0.05

3.4.2 Baseline Result

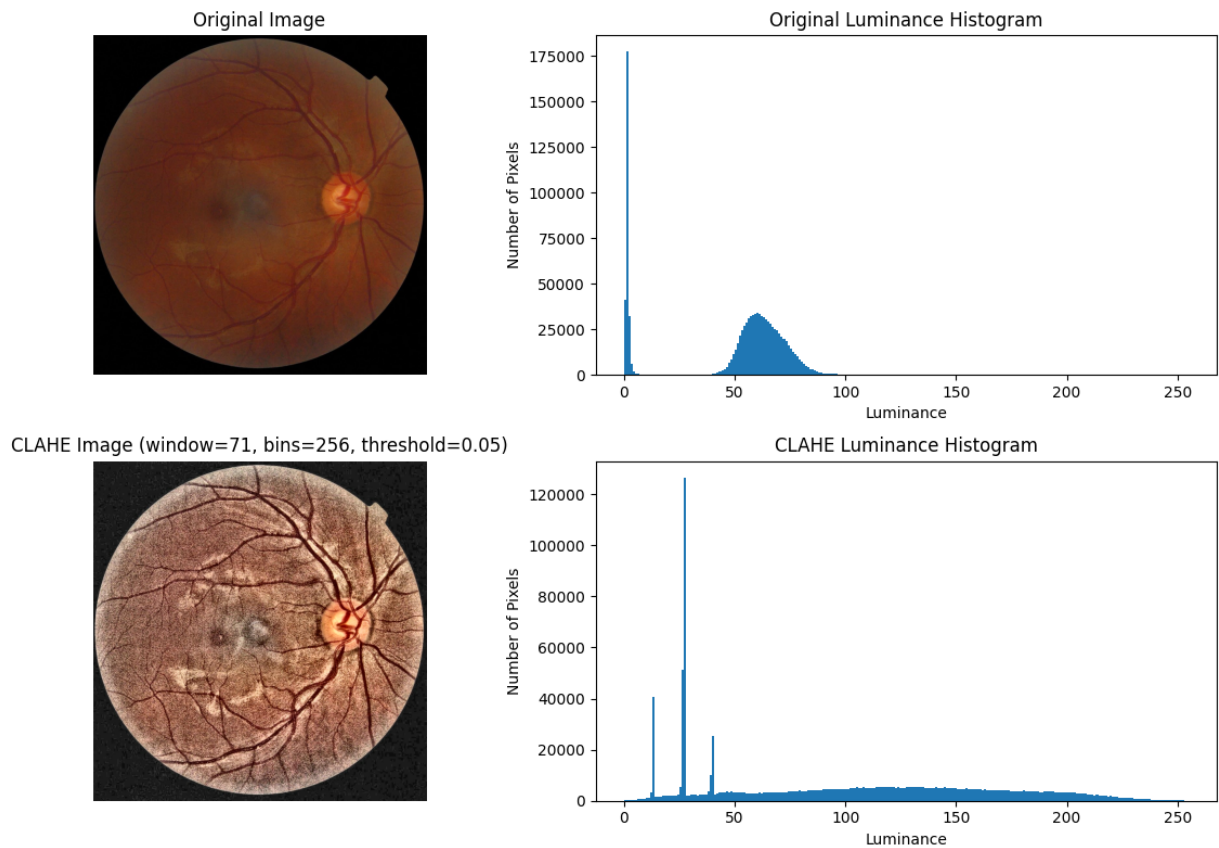


Figure 28: CLAHE on retina.png using the tuned parameters.

Explanation: The selected parameters provide a good balance between enhancing retinal structures and suppressing excessive noise. The blood vessels and fine details become more visible while maintaining a relatively natural appearance.

3.4.3 Significantly Larger Window

Window size: 151

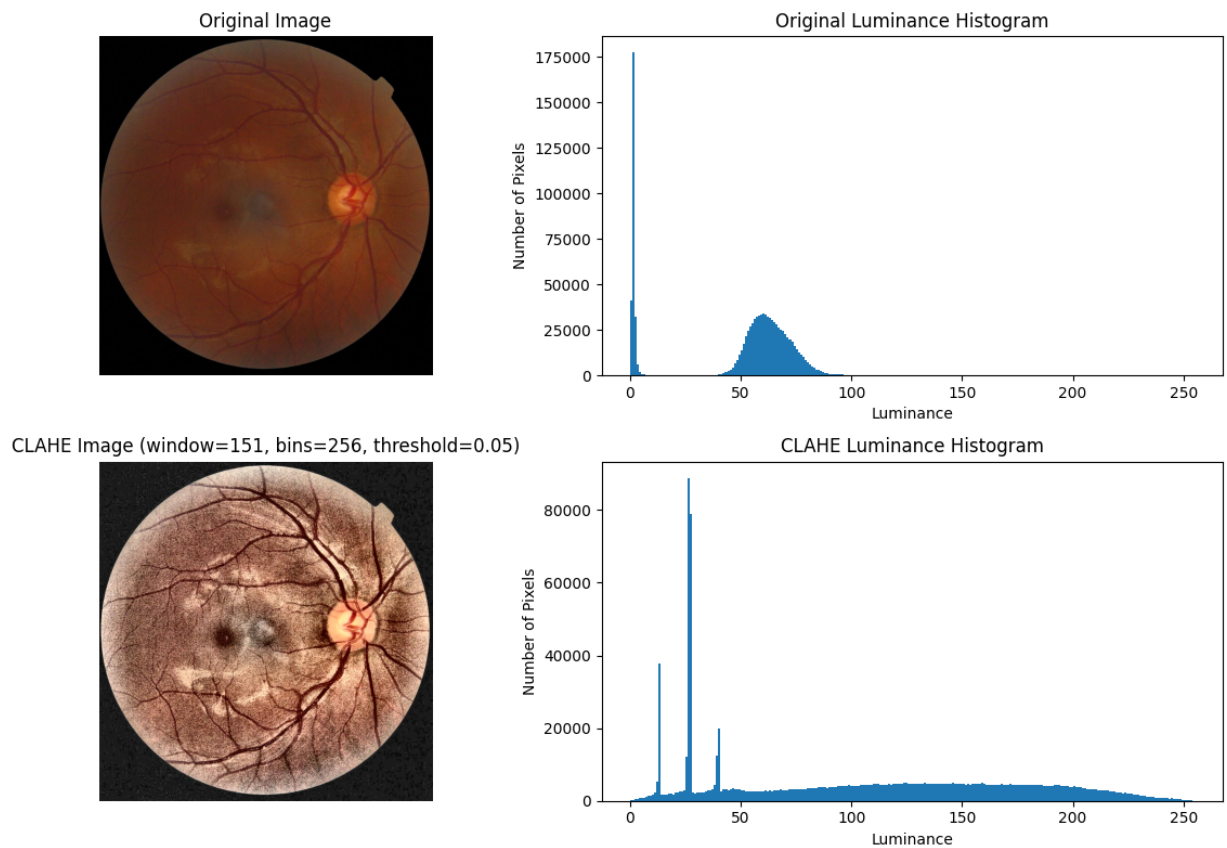


Figure 29: CLAHE on retina.png using a significantly larger window.

Observation: The larger neighborhood produces weaker local contrast enhancement. The result is smoother, but some fine retinal structures are less prominent because the local histogram represents a larger surrounding region.

3.4.4 Significantly Smaller Window

Window size: 31

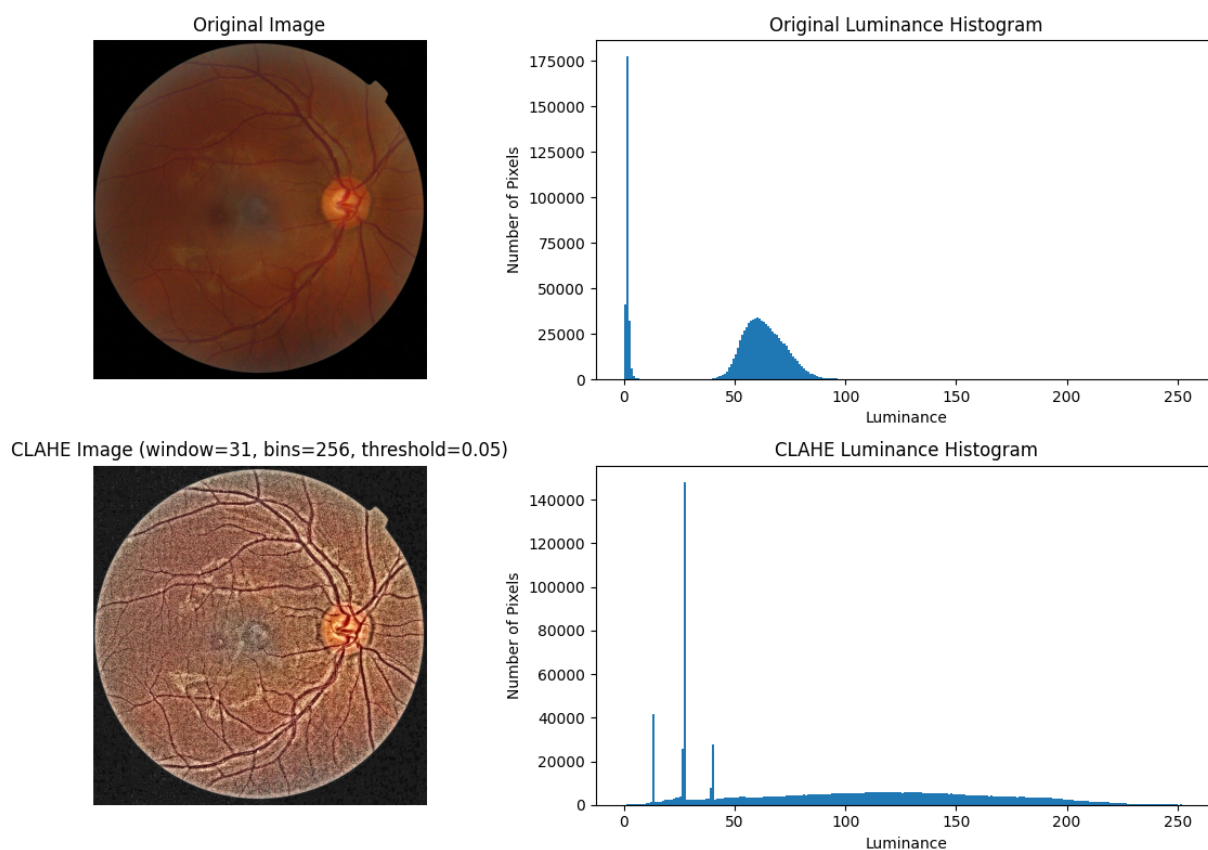


Figure 30: CLAHE on retina.png using a significantly smaller window.

Observation: The smaller neighborhood enhances local details more strongly, making blood vessels and subtle structures more visible. However, it also amplifies noise and produces a grainier appearance.

3.4.5 Half Histogram Threshold

Window size: 71

Original threshold: 0.05

New threshold: 0.025

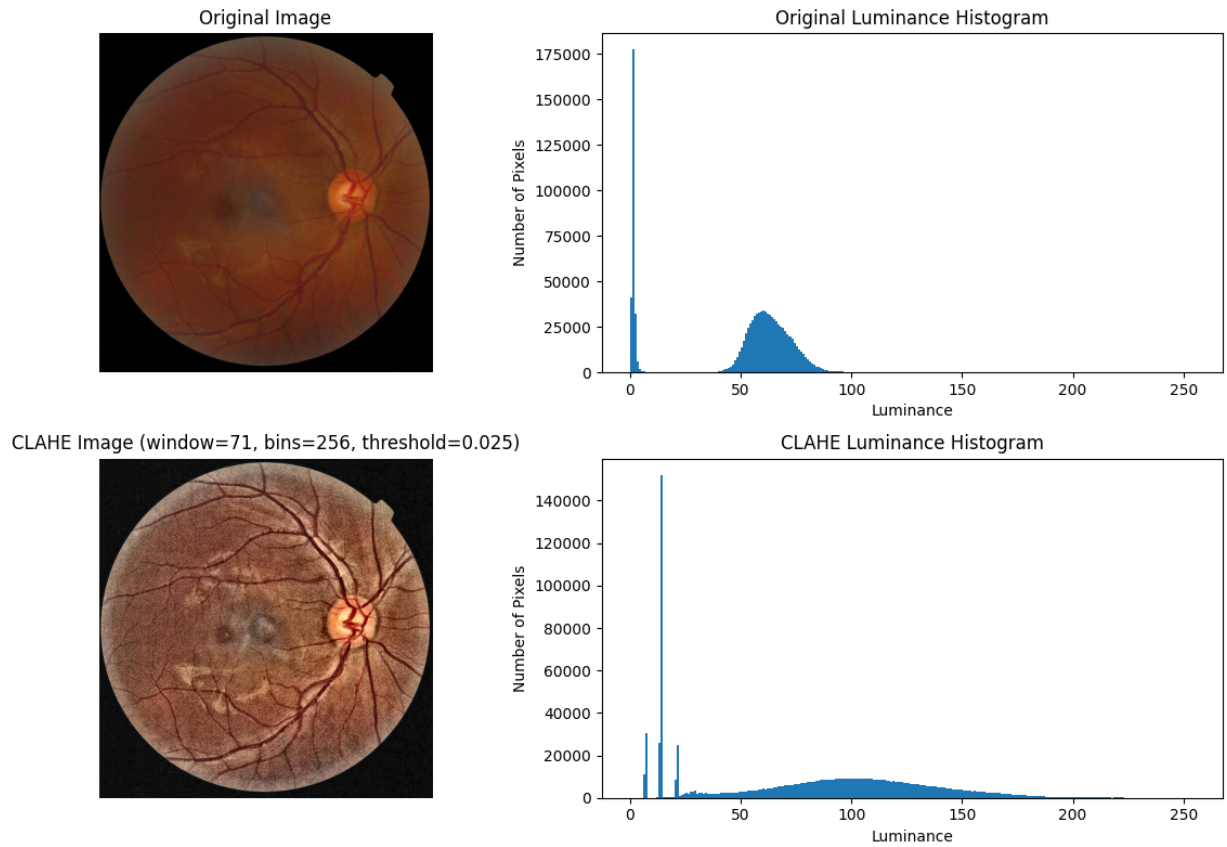


Figure 31: CLAHE on retina.png using half the tuned histogram threshold.

Observation: Reducing the threshold increases the amount of contrast enhancement. More subtle retinal structures become visible, but noise and small intensity variations are also amplified, demonstrating the trade-off between enhancement and noise suppression.

3.5 Histogram Matching

Input image 1: data/hist/retina.png

Input image 2: data/hist/retinaRef.png

Function: myHistMatch()

3.5.1 Significantly Smaller Number of Bins

Number of bins: 32

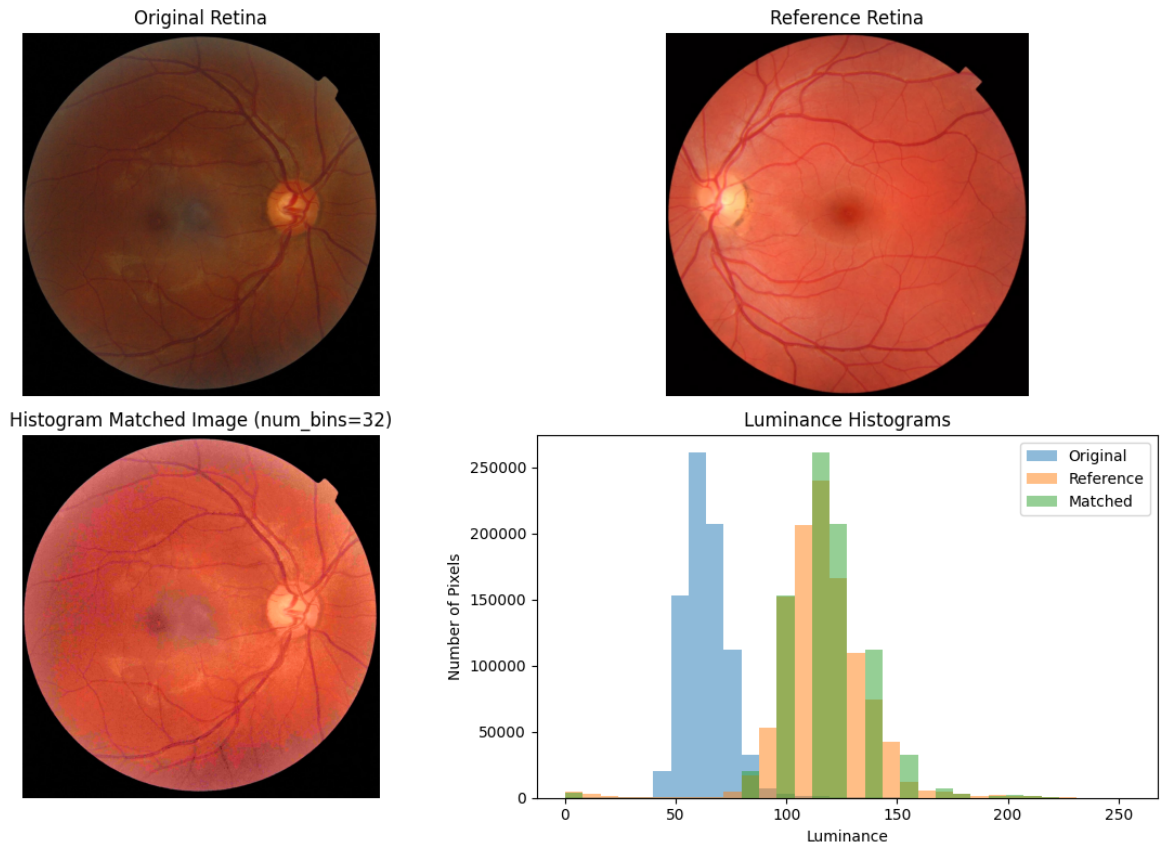


Figure 32: Histogram matching using 32 bins.

What happens when the number of bins is too small? Using only 32 bins produces a coarse representation of the luminance distributions. Important intensity variations are grouped together, resulting in visible banding and less smooth tonal transitions in the matched image. The histogram is therefore not represented with enough detail for accurate matching.

3.5.2 Chosen Number of Bins

Number of bins: 128

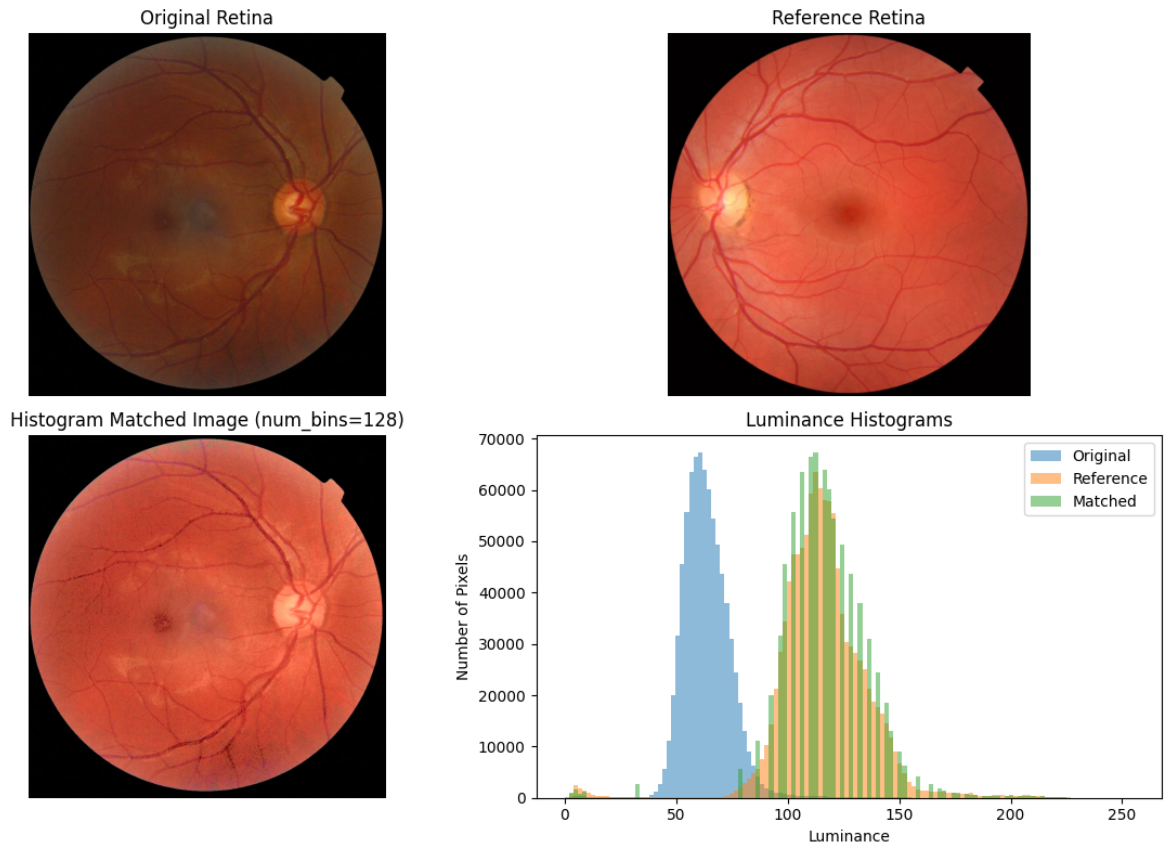


Figure 33: Histogram matching using the chosen 128 bins.

Why was 128 selected as the ideal value? The original retina is darker than the reference image, with its luminance histogram concentrated at lower intensities. Histogram matching shifts the luminance distribution toward that of the reference, producing a brighter image with improved visibility of retinal structures. With 128 bins, the matched histogram follows the reference distribution reasonably well while preserving sufficient intensity detail.

3.5.3 Significantly Larger Number of Bins

Number of bins: 256

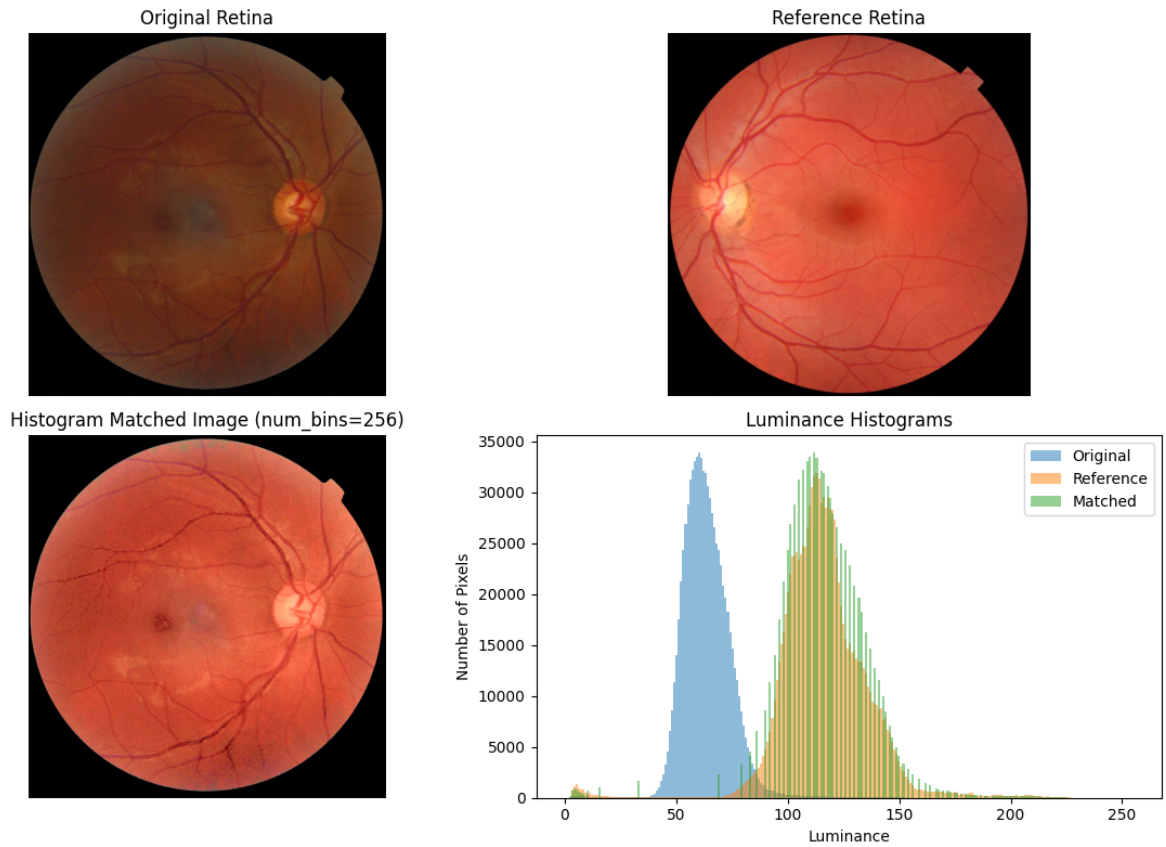


Figure 34: Histogram matching using 256 bins.

What happens when the number of bins is too large? With 256 bins, the histogram captures very fine intensity variations. However, this makes the matching more sensitive to small fluctuations and noise in the image. The resulting histogram becomes more irregular, and the matched image shows less smooth intensity distribution compared with the 128-bin result.

Final conclusion: 128 bins provides the best trade-off between histogram resolution and robustness. It captures the main luminance characteristics of both images without being as coarse as 32 bins or as sensitive to small variations as 256 bins. The resulting image also visually matches the brightness and color characteristics of the reference retina well.